



UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
UTEC

Curso: CLOUD COMPUTING

DISEÑO E IMPLEMENTACIÓN DE UNA ARQUITECTURA CLOUD BASADA EN MICROSERVICIOS PARA ASTRA CLOUD

Informe Técnico del Proyecto

Docente

Colchado Ruíz, Geraldo

Integrantes

Guerrero Cedron, Antonio Jorge
Huaman Huamani, Josue Yeremi
Rosales Esteban, Antony Yonel
Timana Carmona, Jose Manuel
Villalobos Jiménez, José Luis

Lima – Perú
2026

Índice general

1.	Introducción	1
2.	Objetivos	1
2.1.	Objetivo general	1
2.2.	Objetivos específicos	1
3.	Descripción del proyecto	1
3.1.	Problema y solución	1
3.2.	Alcance	1
3.3.	Capa cliente	1
4.	Arquitectura general	2
4.1.	Visión general	2
4.2.	Ruta de acceso	2
5.	Arquitectura de microservicios	3
5.1.	Identity Service	3
5.2.	Catalog Service	3
5.3.	Community Service	3
5.4.	Interaction Service	3
5.5.	Cinema Session Service	3
5.6.	Analytics Service	3
5.7.	Documentación	3
6.	Persistencia políglota	4
6.1.	PostgreSQL	4
6.2.	MySQL	4
6.3.	MongoDB	5
7.	Comunicación entre microservicios	5
8.	Contenedorización	6
9.	Infraestructura AWS	6
9.1.	Red y cómputo	6
9.2.	Exposición de servicios	6
9.3.	S3	6
10.	Frontend web desplegado	6
10.1.	Aplicación en AWS Amplify	6
10.2.	Comunidad y sesión compartida	8
10.3.	Consumo de servicios	9
11.	Disponibilidad y balanceo	10
12.	Pipeline de datos	10
13.	Analítica serverless	11
13.1.	Panel de analítica en la aplicación desplegada	11
13.2.	Consultas analíticas en AWS Athena	12
13.3.	Vistas analíticas en AWS Athena	16
14.	Seguridad	18
14.1.	Identidad y acceso	18
14.2.	Red y configuración	18
14.3.	Consideraciones	18
15.	Infraestructura como código	18
16.	Flujos funcionales	19

16.1. Autenticación y comunidad	19
16.2. Catálogo e interacciones	19
16.3. Sesión y analítica	19
17. Pruebas y validación	19
18. Consideraciones de costos	19
19. Limitaciones actuales	19
20. Mejoras futuras	20
21. Conclusiones	20

1 Introducción

ASTRA CLOUD es una plataforma cinematográfica distribuida desarrollada para el curso de Cloud Computing. El proyecto separa la identidad, el catálogo, la comunidad, las interacciones, las sesiones de cine y la analítica en servicios independientes. Cada servicio se encarga de un dominio específico.

La solución integra APIs REST, WebSocket para las sesiones en vivo, persistencia relacional y documental, contenedores Docker e infraestructura AWS definida como código. También incluye un pipeline que extrae datos operacionales, los guarda en archivos CSV y los deposita en Amazon S3 para analizarlos con Athena.

2 Objetivos

2.1 Objetivo general

Diseñar e implementar una arquitectura cloud basada en microservicios para una plataforma cinematográfica, integrando servicios backend, persistencia polígota, contenedores Docker, infraestructura AWS y procesamiento analítico.

2.2 Objetivos específicos

- Desarrollar servicios independientes para identidad, catálogo, comunidad, interacción social, sesiones y analítica.
- Utilizar bases de datos relacionales y NoSQL de acuerdo con los datos de cada dominio.
- Contenerizar los componentes y definir su despliegue con Docker Compose e infraestructura como código.
- Extraer datos operativos hacia S3 y habilitar consultas analíticas mediante Athena.

3 Descripción del proyecto

3.1 Problema y solución

Una plataforma cinematográfica debe gestionar películas, cuentas, comunidades, reacciones y sesiones compartidas. En ASTRA CLOUD estas tareas se distribuyen entre varios servicios: Catalog administra los metadatos audiovisuales; Identity registra cuentas y emite tokens; Community organiza perfiles, clubes y salas; Interaction registra reseñas, likes, listas e historial; Cinema Session coordina sesiones temporales; y Analytics consulta los datos consolidados.

3.2 Alcance

El workspace contiene las seis APIs de backend, sus configuraciones de contenedor, los procesos de ingesta y la plantilla de infraestructura. REST cubre las operaciones de negocio y WebSocket distribuye eventos de reproducción, membresía y chat dentro de Cinema Session.

3.3 Capa cliente

ASTRA CLOUD cuenta con una aplicación web desplegada mediante AWS Amplify. La interfaz ofrece inicio, exploración del catálogo, detalle de películas, clubes, perfil y órbitas de visualización compartida. Consume las APIs publicadas por API Gateway y los recursos estáticos de S3. El código fuente del frontend y la configuración de Amplify no están incluidos en este workspace, pero la aplicación desplegada se documenta en el Capítulo 10.

4 Arquitectura general

4.1 Visión general

ASTRA CLOUD se organiza en servicios de negocio, almacenamiento operacional y analítica. Las APIs se ejecutan en contenedores, mientras que PostgreSQL, MySQL y MongoDB almacenan la información de cada dominio. Como se observa en la Figura 4.1, la arquitectura integra estos componentes en AWS.

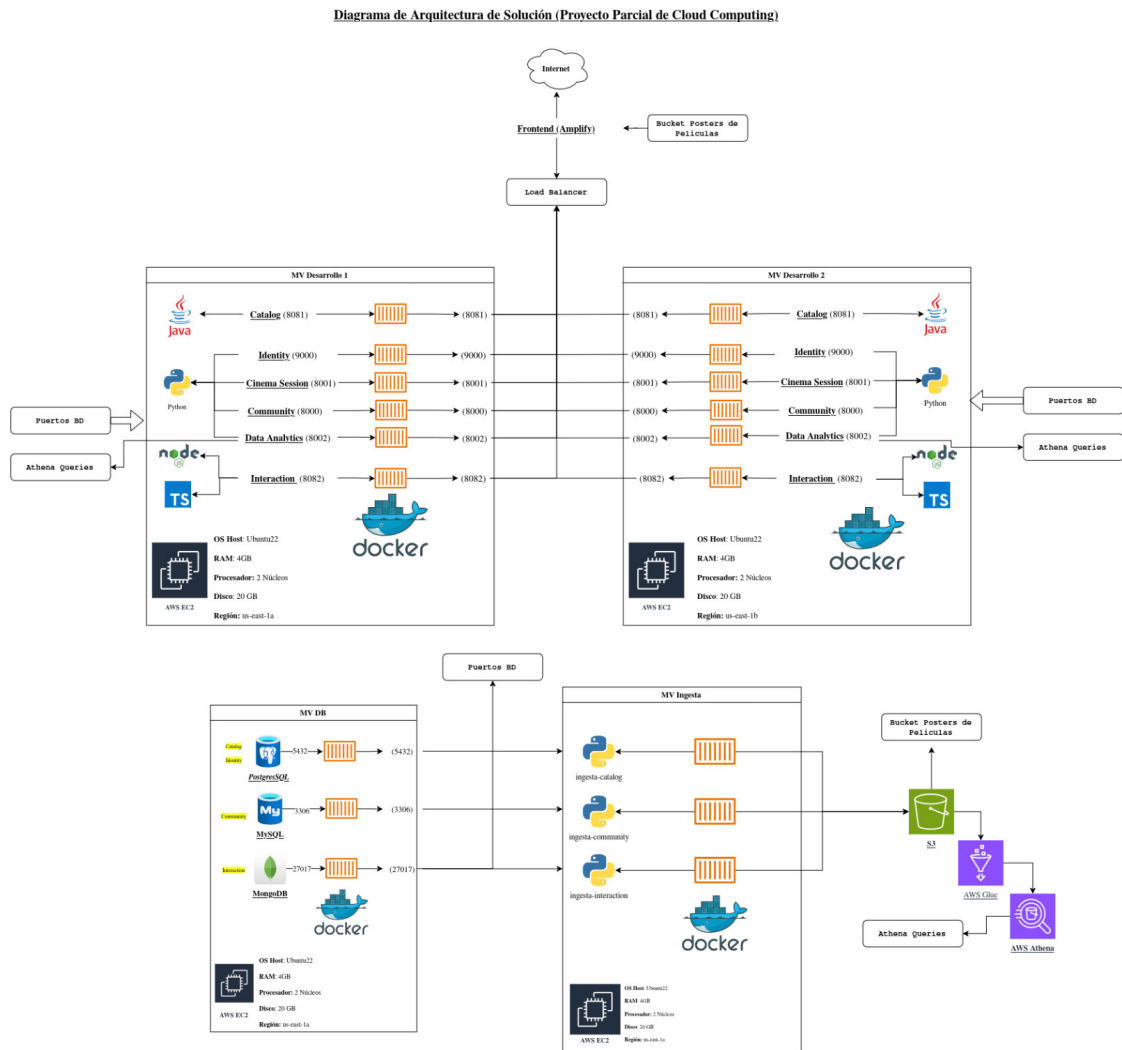


Figura 4.1: Arquitectura general de ASTRA CLOUD

4.2 Ruta de acceso

Para desplegar los servicios se definió la ruta cliente → API Gateway → VPC Link → ALB interno → listener por puerto → grupo destino → EC2 → contenedor. Esta separación centraliza el acceso a las APIs y evita exponer directamente el balanceador.

5 Arquitectura de microservicios

Cuadro 5.1: Microservicios de ASTRA CLOUD

Servicio	Tecnología	Persistencia	Responsabilidad
Identity	Python, FastAPI	PostgreSQL	Registro, autenticación y JWT.
Catalog	Java, Spring Boot	PostgreSQL	Metadatos de películas y reproducción.
Community	Python, FastAPI	MySQL	Perfiles, clubes, membresías y salas.
Interaction	Node.js, Express	MongoDB	Reseñas, likes, listas e historial.
Cinema Session	Python, FastAPI	Memoria	Sesiones grupales, reproducción y chat.
Analytics	Python, FastAPI	Athena	Consultas analíticas sobre S3/Glue.

5.1 Identity Service

Identity expone `/api/auth/register` y `/api/auth/login`. Su tabla `users` almacena UUID, correo único, nombre, hash de contraseña y rol. En el inicio de sesión verifica el hash y emite un JWT HS256 con sub, expiración y emisor `identity-service`.

5.2 Catalog Service

Catalog usa Spring Boot, Spring Data JPA y Flyway. Administra `movie`, `genre`, `artist`, sus relaciones, fuentes de video y subtítulos. Sus rutas principales son `/api/catalog/movies`, `/api/catalog/home`, `/api/catalog/search` y `/api/catalog/movies/{publicId}/session-info`.

5.3 Community Service

Community usa FastAPI, SQLAlchemy y MySQL. Mantiene perfiles asociados al UUID de Identity, sin guardar contraseñas. Sus modelos son `users`, `clubs`, `memberships`, `watch_rooms` y `watch_participants`; las rutas `/api/v1` administran perfiles, clubes, roles y salas persistentes.

5.4 Interaction Service

Interaction emplea Express, Mongoose y MongoDB. Sus colecciones `reviews`, `likes`, `watchlists` y `watch_history` guardan la acción junto con snapshots de usuario y película. El servicio consulta Catalog y Community para validar referencias y formar esos snapshots; `/ingest/*` sirve la extracción paginada.

5.5 Cinema Session Service

Cinema Session mantiene un repositorio en memoria para crear sesiones, participantes y estado de reproducción. El WebSocket `/api/v1/sessions/{session_id}/ws` difunde cambios de reproducción y chat. El estado se libera al finalizar la sesión. Su endpoint de estadísticas consulta Interaction por HTTP.

5.6 Analytics Service

Analytics expone rutas bajo `/api/v1`, como `/movies/top-watched`, `/interactions/movies/best-rated` y `/dashboard/kpis`. Mediante `boto3`, envía SQL a Athena y transforma las filas resultantes en JSON.

5.7 Documentación

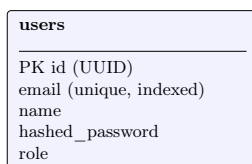
Catalog publica Swagger UI en `/swagger-ui.html`; los servicios FastAPI disponen de `/docs`; Interaction integra Swagger UI en `/docs`.

6 Persistencia políglota

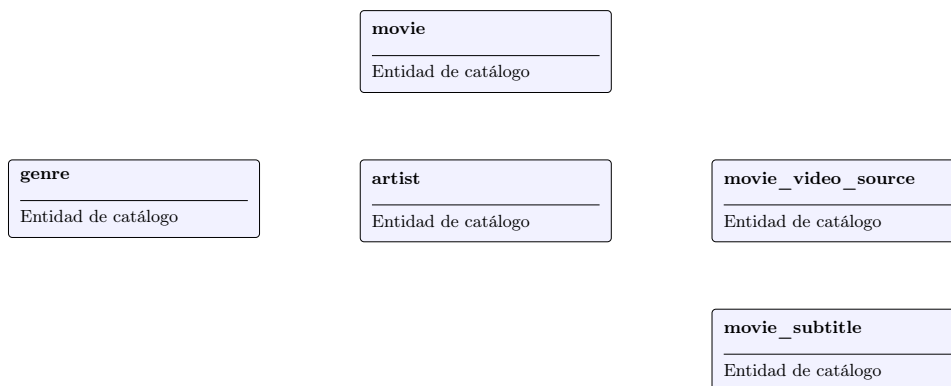
6.1 PostgreSQL

PostgreSQL respalda Identity y Catalog. Identity mantiene **users**; Catalog emplea **movie**, **genre**, **artist**, tablas de relación, fuentes de video y subtítulos. Las relaciones N:M preservan la consistencia entre películas, géneros y artistas, y Flyway versiona el esquema.

POSTGRESQL – IDENTITY



POSTGRESQL – CATALOG



Identity y Catalog conservan esquemas PostgreSQL independientes. Las relaciones entre microservicios se gestionan a nivel de aplicación.

Figura 6.1: Modelo relacional PostgreSQL para Identity y Catalog

6.2 MySQL

Community utiliza MySQL para los modelos de usuarios, clubes, membresías, salas y participantes. La tabla **memberships** materializa la relación usuario–club; las salas guardan anfitrión, película y club opcional. El identificador de película es una referencia de aplicación al catálogo, no una clave foránea interservicio.

MYSQL – COMMUNITY

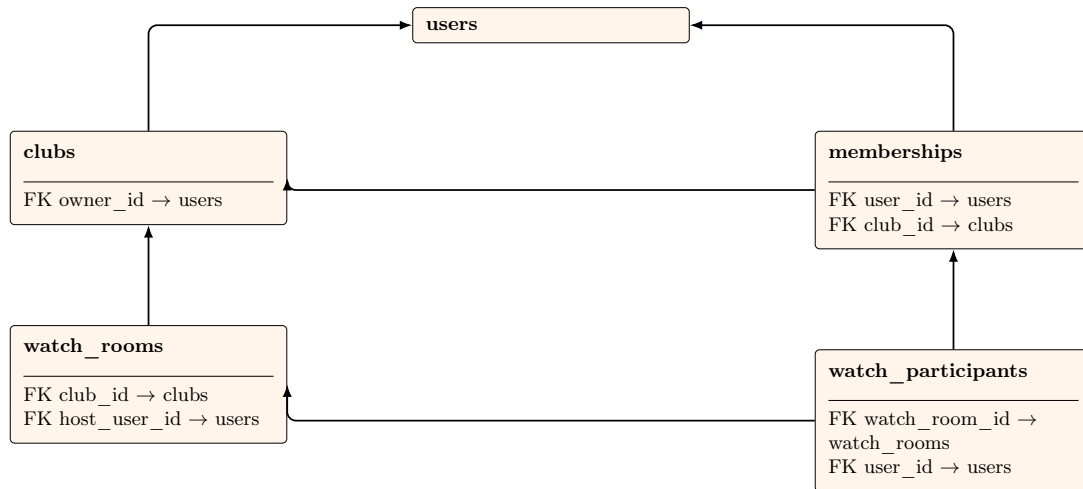


Figura 6.2: Modelo relacional MySQL del Community Service

6.3 MongoDB

Interaction usa MongoDB para **reviews**, **likes**, **watchlists** y **watch_history**. Cada documento combina la acción con snapshots de usuario y película para facilitar la lectura contextual y la extracción analítica. Los índices compuestos únicos de reseñas y likes evitan duplicados por usuario y película.

MONGODB – INTERACTION



Figura 6.3: Modelo documental MongoDB del Interaction Service

7 Comunicación entre microservicios

La comunicación de negocio utiliza HTTP/REST y JWT Bearer en las operaciones protegidas. Identity emite el token; Community, Interaction y Cinema Session validan su firma. Interaction consulta Catalog y Community mediante clientes HTTP configurables para validar referencias y registrar snapshots de película y usuario.

Cuadro 7.1: Integraciones principales

Origen	Destino	Protocolo	Propósito
Interaction	Catalog y Community	HTTP/REST	Validación y snapshots.
Cinema Session	Interaction	HTTP/REST	Estadísticas de película.
Cliente	Cinema Session	WebSocket	Sincronización y chat.
Analytics	Amazon Athena	boto3	Consultas del lago de datos.

8 Contenedorización

Para desplegar los servicios se utilizó Docker, con entornos de ejecución independientes y reproducibles. El Compose de servicios publica Catalog en 8081, Community en 8000, Interaction en 8082, Analytics en 8002, Identity en 9000 y Cinema Session en 8001; todos se conectan mediante una red bridge común. Un Compose separado configura PostgreSQL 16, MySQL 8.4.6 y MongoDB 7 con volúmenes persistentes. Los tres procesos de ingesta también se distribuyen como contenedores y reciben por variables de entorno las conexiones y el destino en S3.

9 Infraestructura AWS

9.1 Red y cómputo

CloudFormation define una VPC 10.0.0.0/16, dos subredes públicas en zonas de disponibilidad diferentes, Internet Gateway, rutas y DNS interno. Incluye dos EC2 de producción, una para bases de datos y otra para ingesta, todas t3.medium con volúmenes gp3 de 20 GiB. La zona privada Route 53 `proycloud.local` resuelve `db.proycloud.local`.

9.2 Exposición de servicios

Un ALB interno contiene listeners y grupos destino para los seis puertos. Seis HTTP API Gateway conectados mediante VPC Link centralizan la entrada. Cada grupo consulta `/health` cada 30 segundos y distribuye tráfico a las dos EC2 de producción.

Cuadro 9.1: Ruteo AWS definido

Servicio	API Gateway	Listener y grupo destino
Identity	IdentityAPI	9000 / TGIdentity
Catalog	CatalogAPI	8081 / TGCatalog
Community	CommunityAPI	8000 / TGCommunity
Interaction	InteractionAPI	8082 / TGInteraction
Analytics	AnalyticsAPI	8002 / TGANalytics
Cinema Session	CinemaAPI	8001 / TGCinema

9.3 S3

La plantilla incorpora un bucket para pósters con cifrado SSE-S3 AES256 y lectura pública de objetos, limitado al contenido visual de películas.

10 Frontend web desplegado

10.1 Aplicación en AWS Amplify

Para ASTRA CLOUD se implementó una aplicación web desplegada mediante AWS Amplify. La interfaz se entrega por HTTPS y permite navegar entre Inicio, Explorar, Clubes, Mis películas, Analítica y las órbitas de reproducción. La Figura 10.1 muestra la portada pública, con contenido destacado, tendencias y accesos a una película o a una experiencia compartida.



Figura 10.1: Inicio público de ASTRA CLOUD desplegado en AWS Amplify.

El catálogo permite explorar las películas y abrir su detalle. La Figura 10.2 muestra la exploración y la Figura 10.3, los metadatos y las acciones disponibles para la película seleccionada, incluida la creación de una órbita, el like y la watchlist. Estas acciones son capacidades protegidas de la interfaz; durante el recorrido no se realizaron operaciones que modificaran los datos de la cuenta de prueba.

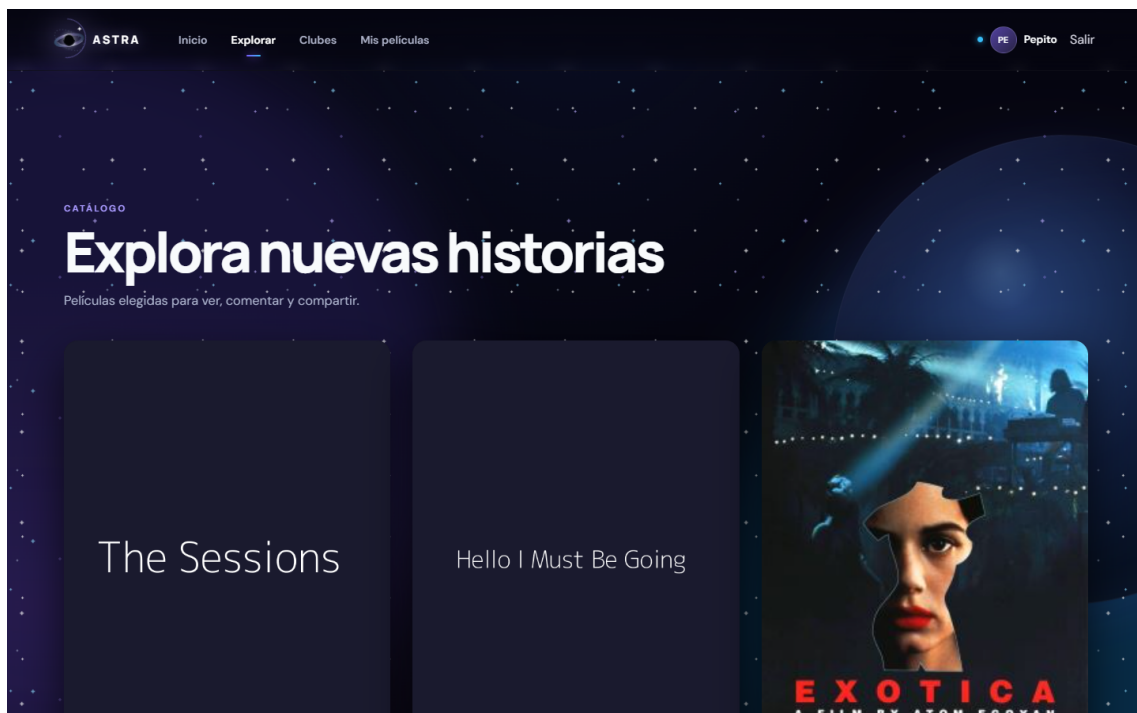


Figura 10.2: Exploración del catálogo cinematográfico desde la aplicación web.

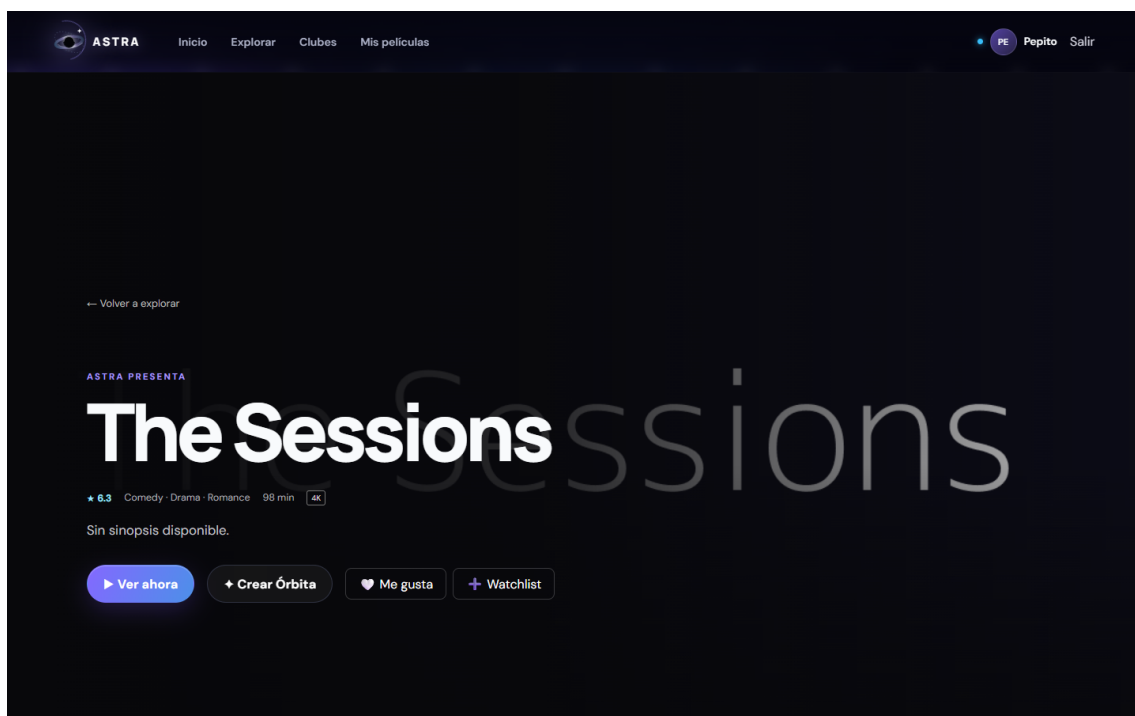


Figura 10.3: Detalle de película con acciones de reproducción, órbita, like y watchlist.

10.2 Comunidad y sesión compartida

La interfaz permite consultar clubes y unirse a una comunidad, como se aprecia en la Figura 10.4. Para el consumo audiovisual compartido, la Figura 10.5 integra el reproductor, los participantes, el chat en vivo y los controles de reacción. Esta pantalla corresponde al dominio Cinema Session y complementa los recursos persistentes de Community.

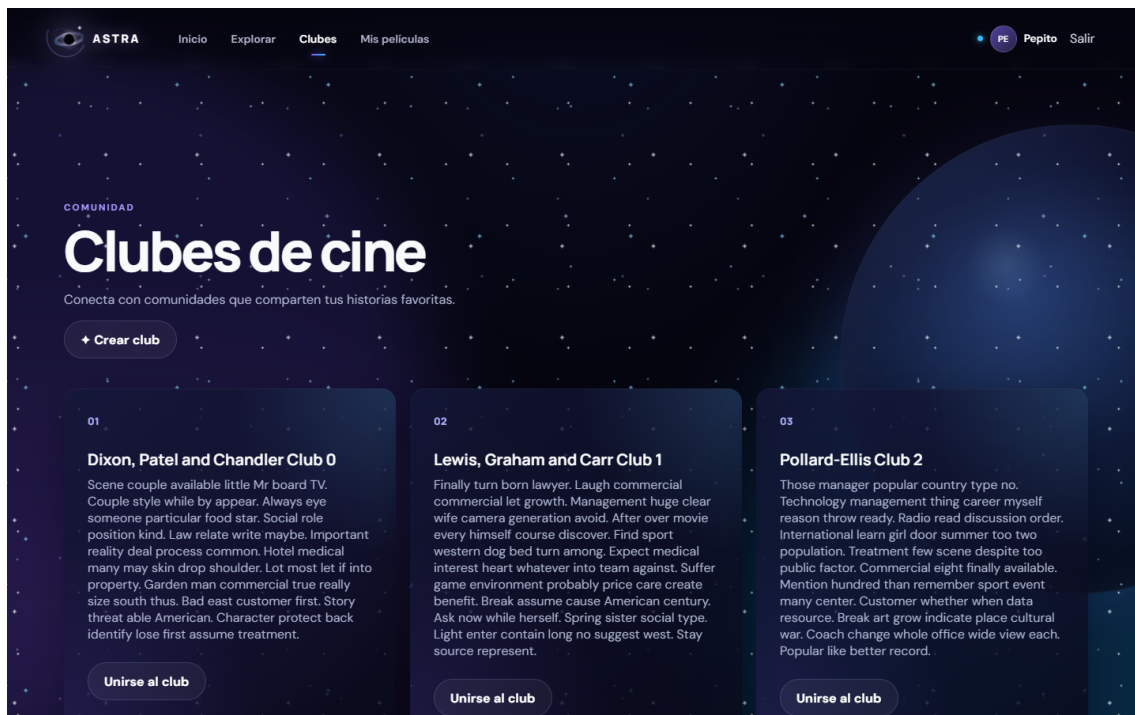


Figura 10.4: Listado de clubes y acceso a las acciones comunitarias.

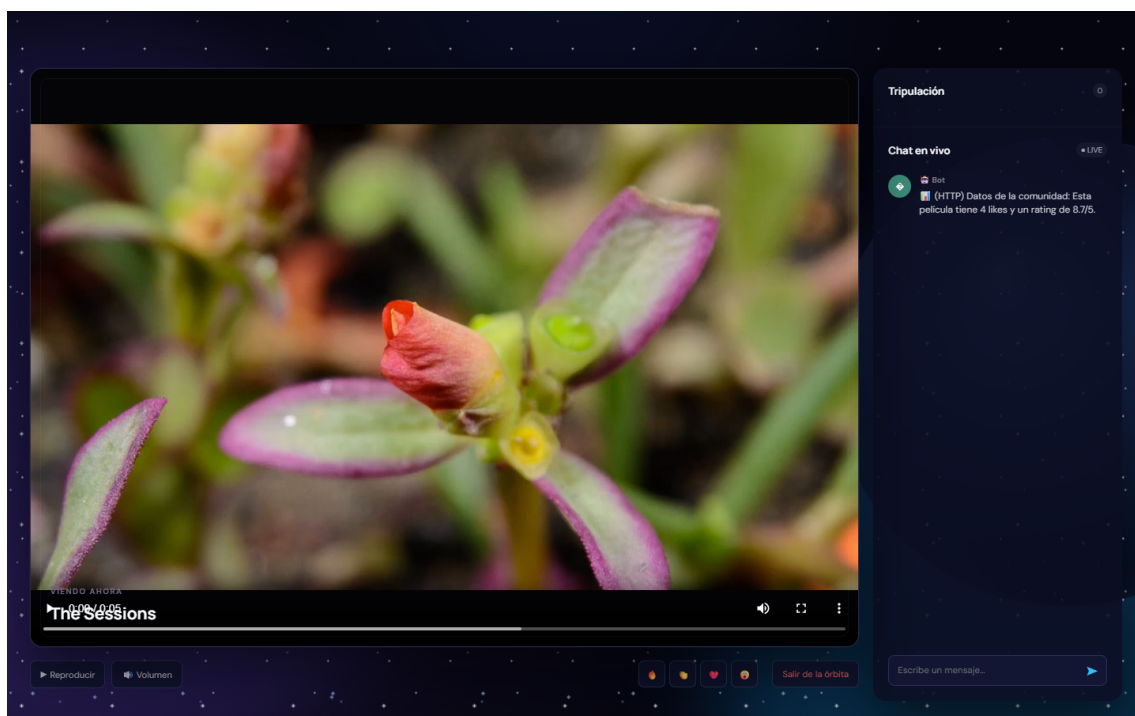


Figura 10.5: Órbita de cine con reproductor, chat en vivo y controles de la sesión.

10.3 Consumo de servicios

El inicio de sesión normal de la interfaz consumió `POST /api/auth/login` y, tras autenticarse, solicitó `GET /api/v1/users/me`. La carga del inicio y la exploración consumió `GET /api/catalog/movies`. La navegación por detalle, clubes y sesión confirmó la integración visual con sus dominios correspondientes. La Tabla 10.1 distingue las operaciones confirmadas durante el recorrido de las capacidades expuestas por el backend; no se atribuyen a la interfaz operaciones que no se ejecutaron.

Cuadro 10.1: Relación entre frontend y microservicios.

Servicio	Funcionalidad del frontend	Operaciones/API comprobadas o disponibles en la implementación
Identity	Inicio de sesión	POST <code>/api/auth/login</code> comprobado en la aplicación; POST <code>/api/auth/register</code> disponible en el servicio.
Catalog	Inicio, explorar y detalle de película	GET <code>/api/catalog/movies</code> comprobado; GET <code>/api/catalog/movies/{publicId}</code> disponible para detalle.
Community	Perfil y clubes	GET <code>/api/v1/users/me</code> comprobado; GET <code>/api/v1/clubs</code> disponible para el listado de clubes.
Interaction	Like, watchlist y reseñas	Acciones visibles en el detalle; POST/DELETE <code>/movies/{movieId}/like</code> y GET/POST <code>/users/{userId}/watchlist</code> están implementados. No se invocaron operaciones mutables.
Cinema Session	Órbita, reproducción y chat	Interfaz de sesión recorrida; GET/POST <code>/api/v1/sessions</code> y POST <code>/api/v1/sessions/{id}/chat</code> están implementados.
Analytics	Panel <code>/analytics</code> con KPIs y tablas	GET <code>/api/v1/dashboard/kpis</code> comprobado durante el recorrido; la evidencia del panel se presenta en la Figura 13.1.

La tabla muestra que Catalog, Community, Interaction, Cinema Session y Analytics disponen de capacidades REST integradas en la aplicación. El recorrido confirmó de forma directa el inicio de sesión, las lecturas de catálogo y perfil, y la consulta de indicadores de Analytics. Las acciones que alteran datos no se ejecutaron para preservar la cuenta de prueba; por ello, el consumo de al menos dos métodos REST desde el frontend no queda completamente demostrado para cada microservicio con este recorrido.

11 Disponibilidad y balanceo

La arquitectura coloca dos instancias de producción en subredes de zonas de disponibilidad distintas y las registra en los seis grupos destino del ALB. Los health checks retiran del ruteo una réplica que no responda correctamente. No se definió un Auto Scaling Group; la redundancia contemplada es de dos réplicas estáticas. La base de datos y la ingesta se ejecutan en instancias únicas. Cinema Session conserva estado por proceso, por lo que una sesión no se comparte automáticamente entre réplicas.

12 Pipeline de datos

El pipeline utiliza tres contenedores Python. `ingesta-catalogo` extrae tablas de PostgreSQL; `ingesta-comunity`, tablas de MySQL; y `ingesta-interaction`, colecciones de MongoDB. Cada proceso genera archivos CSV en UTF-8, limpia los saltos de línea y los carga con `boto3` a un bucket S3 configurable.

Cuadro 12.1: Datos extraídos hacia S3

Origen	Prefijo S3	Datos
Catalog/PostgreSQL	resultados/ catalogo	Películas, artistas, géneros, relaciones, videos y subtítulos.
Community/MySQL	resultados/ community	Usuarios, clubes, membresías, salas y participantes.
Interaction/MongoDB	resultados/ interaction	Colecciones disponibles, incluidas reseñas, likes y watchlists.

Las claves incluyen la entidad y la partición YYYY-MM-DD. Esta estructura prepara los archivos para su catalogación en Glue y consulta en Athena, sin cargar las bases transaccionales. La programación periódica y los crawlers de Glue se configuran como parte del despliegue del lago de datos.

13 Analítica serverless

Analytics se implementó como una API FastAPI que delega consultas en Amazon Athena. Su cliente inicia una ejecución, realiza *polling* hasta **SUCCEEDED**, **FAILED** o **CANCELLED**, y devuelve las filas como JSON. La ubicación de resultados se configura mediante **ATHENA_OUTPUT_BUCKET**.

Los datos operacionales de ASTRA CLOUD pertenecen a dominios diferentes: el catálogo se almacena en PostgreSQL, la información social en MySQL y las interacciones en MongoDB. Los contenedores de ingesta los extraen en archivos CSV y los depositan en Amazon S3. AWS Glue permite catalogar esas estructuras y Athena ejecuta SQL sobre la información consolidada, por lo que se pueden realizar análisis transversales sin trasladar carga de consulta a los microservicios transaccionales.

13.1 Panel de analítica en la aplicación desplegada

El cliente web expone la ruta **/analytics** para usuarios autenticados. Durante el recorrido de validación, el panel solicitó los indicadores de **/api/v1/dashboard/kpis** y las vistas de géneros, actores y clubes; los resultados se presentan sin ejecutar operaciones que modifiquen los datos.

La Figura 13.1 muestra el resultado observado. El panel concentra los conteos de películas, usuarios, clubes, salas e interacciones, junto con las tablas de géneros más vistos, actores populares y clubes activos. Esta vista conecta la API de analítica implementada con un consumo visible en la aplicación: los datos consolidados se traducen en indicadores que un administrador puede consultar desde la interfaz.

La misma evidencia permite identificar una limitación del despliegue actual: el campo *Rating Promedio* aparece como **NaN**. Por ello, no se interpreta ese valor como una métrica válida en este informe; su corrección requiere revisar el dato retornado o su transformación en el cliente.

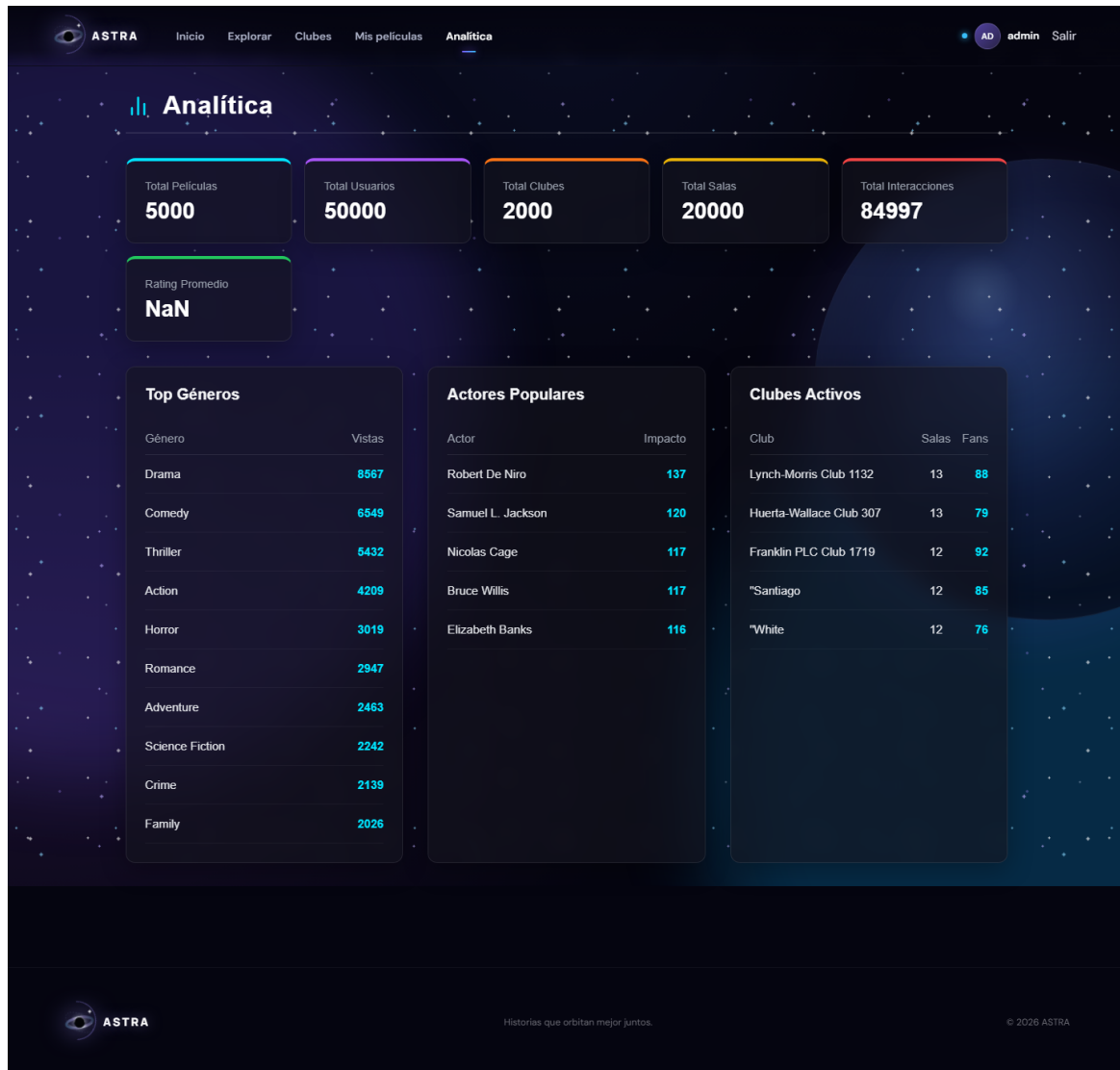


Figura 13.1: Panel de analítica desplegado con KPIs y tablas de géneros, actores y clubes.

13.2 Consultas analíticas en AWS Athena

Consulta 1: Engagement por género

Para analizar el nivel de participación por categoría cinematográfica se desarrolló la siguiente consulta. Integra `genre`, `movie_genre`, `movie`, `watch_rooms` e `interaction`; así obtiene el total de películas, salas de visualización, rating promedio, reseñas y likes para cada género.

```
WITH movie_stats AS (
  SELECT m.id as movie_id, m.rating as avg_rating,
    (SELECT COUNT(DISTINCT id)
     FROM "astra_analytics"."watch_rooms"
     WHERE movie_id = CAST(m.public_id AS VARCHAR)) as watch_rooms,
    (SELECT COUNT(1) FROM "astra_analytics"."interaction"
     WHERE movie_id = CAST(m.public_id AS VARCHAR)
     AND partition_0 = 'reviews') as total_reviews,
    (SELECT COUNT(1) FROM "astra_analytics"."interaction"
     WHERE movie_id = CAST(m.public_id AS VARCHAR)
     AND partition_0 = 'likes') as total_likes
  FROM "astra_analytics"."movie" m
)
SELECT g.name as genre_name, COUNT(DISTINCT mg.movie_id) as total_movies,
  SUM(ms.watch_rooms) as total_watch_rooms,
```

```

    AVG(ms.avg_rating) as avg_rating, SUM(ms.total_reviews) as total_reviews,
    SUM(ms.total_likes) as total_likes
FROM "astra_analytics"."genre" g
JOIN "astra_analytics"."movie_genre" mg ON g.id = mg.genre_id
JOIN movie_stats ms ON mg.movie_id = ms.movie_id
GROUP BY g.name
ORDER BY total_watch_rooms DESC;

```

Listing 13.1: Consulta SQL de engagement por género en Athena

El CTE `movie_stats` calcula las métricas por película antes de agruparlas por género. Las subconsultas relacionan el identificador público del catálogo con los registros consolidados de salas e interacciones, mientras que la consulta exterior suma y promedia los indicadores por categoría.

La Figura 13.2 presenta la ejecución completada en Athena. En las filas visibles, Drama concentra 8 567 *watch rooms*, 8 495 reseñas y 10 651 likes; Comedy aparece a continuación con 6 549 salas. Estos resultados permiten priorizar el análisis de actividad de las categorías con mayor participación.

Query results

Query stats

Completed

Time in queue: 91 ms

Run time: 3.02 sec

Data scanned: 16.03 MB

Results (20)

Search rows

<1>

⚙

Copy

Download results CSV

#	genre_name	total_movies	total_watch_rooms	avg_rating	total_reviews	total_likes
1	Drama	2119	8567	471.3651532349603	8495	10651
2	Comedy	1652	6549	496.37054698457223	6724	8234
3	Thriller	1350	5432	428.68	5324	6853
4	Action	1036	4209	543.1036674816627	4017	5170
5	Horror	763	3019	470.30212121212116	3037	3861
6	Romance	735	2947	522.461872909699	2919	3620
7	Adventure	613	2463	547.9900793650794	2408	3080
8	Science Fiction	550	2242	582.2607655502393	2178	2782
9	Crime	528	2139	495.9339285714285	2013	2581
10	Family	524	2076	532.2789719676169	2138	2572

Figura 13.2: Resultado de la consulta de engagement por género en AWS Athena

Consulta 2: Participación de usuarios en la plataforma

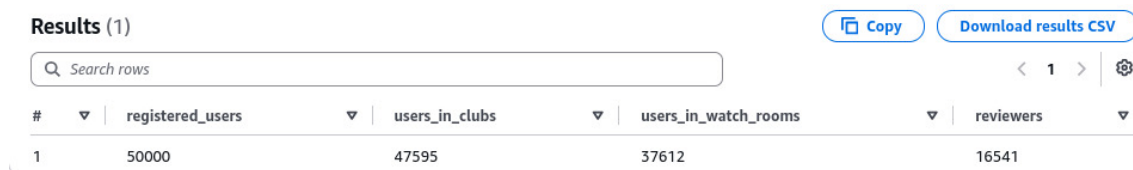
Para obtener una visión consolidada de participación se desarrolló una consulta que compara la población registrada con los usuarios únicos que intervienen en clubes, *watch rooms* y reseñas. Se utilizan las tablas `users`, `memberships`, `watch_participants` e `interaction`. `COUNT(1)` mide el total registrado, mientras que `COUNT(DISTINCT ...)` evita contar más de una vez a un usuario que participa repetidamente en una funcionalidad.

```
SELECT
  (SELECT COUNT(1) FROM "astra_analytics"."users") AS registered_users ,
  (SELECT COUNT(DISTINCT user_id)
   FROM "astra_analytics"."memberships") AS users_in_clubs ,
  (SELECT COUNT(DISTINCT user_id)
   FROM "astra_analytics"."watch_participants") AS users_in_watch_rooms ,
  (SELECT COUNT(DISTINCT user_id)
   FROM "astra_analytics"."interaction"
   WHERE partition_0 = 'reviews') AS reviewers;
```

Listing 13.2: Consulta SQL de participación de usuarios en Athena

La consulta concentra cuatro agregaciones independientes en una única fila de resultado. Esta forma de presentación facilita la comparación de la base de usuarios con su participación en las funcionalidades sociales de ASTRA CLOUD.

Como se aprecia en la Figura 13.3, Athena devuelve 50 000 usuarios registrados, 47 595 usuarios en clubes, 37 612 participantes en *watch rooms* y 16 541 revisores. La consulta proporciona una referencia directa de la presencia de usuarios en cada etapa considerada.



#	registered_users	users_in_clubs	users_in_watch_rooms	reviewers
1	50000	47595	37612	16541

Figura 13.3: Resultado de la consulta de participación de usuarios en AWS Athena

Consulta 3: Métricas de interacción por película

Para integrar el catálogo con las interacciones y las salas de visualización se construyó una consulta analítica por película. Participan `movie`, `interaction` y `watch_rooms`; el resultado incluye identificador público, título, rating de catálogo, promedio de puntuaciones de usuarios, likes y cantidad de salas asociadas.

```
WITH movie_metrics AS (
  SELECT m.public_id, m.title, m.rating AS catalog_rating,
         (SELECT AVG(CAST(score AS DOUBLE))
          FROM "astra_analytics"."interaction"
          WHERE movie_id = CAST(m.public_id AS VARCHAR)
          AND partition_0 = 'reviews') AS avg_user_review_score,
         (SELECT COUNT(1) FROM "astra_analytics"."interaction"
          WHERE movie_id = CAST(m.public_id AS VARCHAR)
          AND partition_0 = 'likes') AS likes_count,
         (SELECT COUNT(1) FROM "astra_analytics"."watch_rooms"
          WHERE movie_id = CAST(m.public_id AS VARCHAR)) AS watch_room_count
  FROM "astra_analytics"."movie" m
)
SELECT * FROM movie_metrics
ORDER BY watch_room_count DESC
LIMIT 100;
```

Listing 13.3: Consulta SQL de métricas de interacción por película en Athena

El CTE `movie_metrics` construye primero los indicadores por película. Posteriormente, Athena ordena las filas por `watch_room_count` de forma descendente y limita la salida a las primeras 100 películas, manteniendo una vista manejable para el consumo de la API analítica.

La Figura 13.4 registra una ejecución completada con 100 resultados. La captura muestra, entre las primeras filas, títulos como *Contracted: Phase II*, *Getting Even with Dad* y *Rise of the Guardians*, junto con sus identificadores públicos y los campos de métricas. Esta vista permite relacionar directamente los datos del catálogo con la actividad registrada en la plataforma.

#	public_id	title	catalog_rating	avg_user_review_score
1	fce67fb3-d5c8-42d0-9f23-4edaa2ab7d7b	Contracted: Phase II	2015.0	
2	0a4dfcda-382f-4705-9110-bc2a6d85a77f	Getting Even with Dad		
3	e57e54de-05df-4db3-974f-0aecc399d6b6	Rise of the Guardians		
4	417a9ede-2675-4f9c-abd2-0e7192066d89	Miss Austen Regrets	90.0	
5	f4f2b3fa-9de1-4d57-8536-edfa48d5e09f	Compliance		
6	8260ee0c-3076-474b-9e50-9a77883194f5	Vanishing on 7th Street	2010.0	
7	fb3bc234-fc51-48dc-9a80-af621efdeb26	Jack of the Red Hearts	7.0	
8	1edab6bb-813b-48e7-9c65-ada51a7057d6	Back in Time		
9	bac4527e-1057-4097-8343-d7c2e2b4ebd2	Sicario		
10	591625c8-b1da-4372-8866-6b7da86dc745	Excision	2012.0	
11	868cf08a-1d4a-4b56-b80a-09fb55deb9ff	Sorority Boys	93.0	

Figura 13.4: Resultado de la consulta de métricas de interacción por película en AWS Athena

Consulta 4: Nivel de actividad de los clubes

Para analizar la actividad de las comunidades de ASTRA CLOUD se desarrolló una consulta que relaciona los clubes con sus miembros, las salas creadas y la diversidad de películas visualizadas. Participan las tablas `clubs`, `memberships` y `watch_rooms`; el resultado incluye el nombre del club, miembros únicos, salas, relación de actividad y películas diferentes asociadas.

```
SELECT c.name AS club_name,
       COUNT(DISTINCT m.user_id) AS member_count,
       COUNT(DISTINCT wr.id) AS total_watch_rooms,
       CAST(COUNT(DISTINCT wr.id) AS DOUBLE)
       / NULLIF(COUNT(DISTINCT m.user_id), 0) AS activity_ratio,
       COUNT(DISTINCT wr.movie_id) AS unique_movies_watched
FROM "astra_analytics"."clubs" c
LEFT JOIN "astra_analytics"."memberships" m ON c.id = m.club_id
LEFT JOIN "astra_analytics"."watch_rooms" wr ON c.id = wr.club_id
GROUP BY c.name
ORDER BY activity_ratio DESC NULLS LAST;
```

Listing 13.4: Consulta SQL de nivel de actividad de clubes en Athena

Los `LEFT JOIN` conservan los clubes aunque todavía no tengan membresías o salas asociadas. A su vez, `NULLIF` evita una división entre cero cuando un club no registra miembros; esto permite ordenar la actividad sin excluir comunidades de la visión analítica.

La Figura 13.5 muestra 1712 resultados ordenados por `activity_ratio`. En las filas visibles, Townsend-Wood Club 1456 registra 53 miembros, 11 salas y una relación de actividad de

aproximadamente 0.208; Huerta-Wallace Club 307 registra 79 miembros y 13 salas. La consulta permite comparar la creación de salas con el tamaño de cada comunidad.

Completed Time in queue: 112 ms Run time: 2.201 sec Data scanned: 10.07 MB

Results (1,712) [Copy](#) [Download results CSV](#)

Search rows

#	club_name	member_count	total_watch_rooms	activity_ratio	unique_movies_watched
1	Townsend-Wood Club 1456	53	11	0.20754716981132076	11
2	"Conley	66	12	0.18181818181818182	12
3	Parker Inc Club 14	62	11	0.1774193548387097	11
4	Willis Group Club 1497	58	10	0.1724137931034483	10
5	"Mayer	65	11	0.16923076923076924	11
6	Huerta-Wallace Club 307	79	13	0.16455696202531644	13
7	Alexander-Moses Club 1576	68	11	0.16176470588235295	11
8	Mcmillan PLC Club 1215	68	11	0.16176470588235295	11
9	Keller-Mcdonald Club 1213	62	10	0.16129032258064516	10
10	Martinez-Bowman Club 350	63	10	0.15873015873015872	10

Figura 13.5: Resultado de la consulta de actividad de clubes en AWS Athena

13.3 Vistas analíticas en AWS Athena

Además de las consultas analíticas, se implementaron vistas en Athena para encapsular integraciones que se reutilizan dentro de ASTRA CLOUD. Cada vista reúne en una estructura consultable los datos que originalmente pertenecen a distintos componentes de la plataforma.

Vista 1: Consolidado analítico de películas

La vista `vw_consolidado_peliculas` crea una representación reutilizable del catálogo, sus géneros, las salas de visualización y las interacciones. Proporciona identificador de película, título, género, total de salas, total de likes y promedio de puntuaciones de usuarios a partir de `movie`, `movie_genre`, `genre`, `watch_rooms` e `interaction`.

```
CREATE OR REPLACE VIEW vw_consolidado_peliculas AS
SELECT m.public_id AS movie_id, m.title, g.name AS genre,
       COUNT(DISTINCT w.id) AS total_salas,
       COUNT(i_likes.user_id) AS total_likes,
       AVG(CAST(i_reviews.score AS DOUBLE)) AS promedio_usuarios
FROM "astra_analytics"."movie" m
LEFT JOIN "astra_analytics"."movie_genre" mg ON m.id = mg.movie_id
LEFT JOIN "astra_analytics"."genre" g ON mg.genre_id = g.id
LEFT JOIN "astra_analytics"."watch_rooms" w
      ON CAST(m.public_id AS VARCHAR) = w.movie_id
LEFT JOIN "astra_analytics"."interaction" i_likes
      ON CAST(m.public_id AS VARCHAR) = i_likes.movie_id
      AND i_likes.partition_0 = 'likes'
LEFT JOIN "astra_analytics"."interaction" i_reviews
      ON CAST(m.public_id AS VARCHAR) = i_reviews.movie_id
      AND i_reviews.partition_0 = 'reviews'
GROUP BY m.public_id, m.title, g.name;
```

Listing 13.5: Creación de la vista `vw_consolidado_peliculas`

La Figura 13.6 registra la creación exitosa de la vista y la Figura 13.7 presenta su consulta. En las filas visibles aparecen *Jurassic World*, *Mad Max: Fury Road* y *Terminator Genisys*, con

sus géneros, salas y likes consolidados. La vista evita reconstruir manualmente estas relaciones en cada análisis posterior.



Figura 13.6: Creación de la vista vw_consolidado_peliculas en AWS Athena

#	movie_id	title	genre	total_salas	total_likes
1	fdcd1319-aa5f-4d21-807f-9f32b1766804	Jurassic World	Science Fiction	4	20
2	ecb6a464-9acf-44d8-9af4-66e62a425052	Mad Max: Fury Road	Science Fiction	5	45
3	89339279-ceb3-412b-ae0c-55fdb40893ad	Terminator Genisys	Thriller	3	66

Figura 13.7: Consulta de la vista vw_consolidado_peliculas en AWS Athena

Vista 2: Perfil social de usuarios

La vista vw_perfil_social_usuarios consolida la participación social de cada usuario. Relaciona users, memberships y watch_participants para ofrecer identificador, username, correo electrónico, clubes unidos y salas en las que participó.

```
CREATE OR REPLACE VIEW vw_perfil_social_usuarios AS
SELECT u.id AS user_id, u.username, u.email,
       COUNT(DISTINCT m.club_id) AS clubes_unidos,
       COUNT(DISTINCT wp.watch_room_id) AS salas_participadas
FROM "astra_analytics"."users" u
LEFT JOIN "astra_analytics"."memberships" m
  ON CAST(u.id AS VARCHAR) = CAST(m.user_id AS VARCHAR)
LEFT JOIN "astra_analytics"."watch_participants" wp
  ON CAST(u.id AS VARCHAR) = CAST(wp.user_id AS VARCHAR)
GROUP BY u.id, u.username, u.email;
```

Listing 13.6: Creación de la vista vw_perfil_social_usuarios

La creación de la vista se muestra en la Figura 13.8 y su consulta en la Figura 13.9. Las filas visibles incluyen usuarios con diferentes combinaciones de clubes y salas, como jeanmoreno_3781 con 4 clubes y 4 salas. Esta representación resume la presencia de cada usuario en las funcionalidades sociales sin repetir los conteos en cada consulta de consumo.



Figura 13.8: Creación de la vista vw_perfil_social_usuarios en AWS Athena

#	user_id	username	email	clubes_unidos	salas_participadas
1	3782	jeanmoreno_3781	3781_harrisbrian@example.com	4	4
2	3790	jodi93_3789	3789_mcintoshbenjamin@example.net	5	2
3	3811	aguilardiane_3810	3810_brian93@example.org	4	2
4	3831	jason84_3830	3830_tony09@example.net	4	0
5	3836	oalexander_3835	3835_vodom@example.org	3	5
6	3853	jasminebrown_3852	3852_sheltonlaura@example.com	6	3

Figura 13.9: Consulta de la vista vw_perfil_social_usuarios en AWS Athena

14 Seguridad

14.1 Identidad y acceso

Identity se encarga del registro y del inicio de sesión. Las contraseñas se guardan como hashes y el servicio emite JWT HS256. Community, Interaction y Cinema Session validan el token antes de atender rutas protegidas, sin almacenar las contraseñas de los usuarios.

14.2 Red y configuración

La plantilla define grupos de seguridad para ALB, producción y bases de datos. PostgreSQL, MySQL y MongoDB aceptan conexiones desde el grupo de producción. Secretos y contraseñas se suministran como parámetros NoEcho y variables de entorno; el bucket de pósters cuenta con cifrado SSE-S3.

14.3 Consideraciones

Para el entorno académico se definieron subredes públicas, SSH abierto y lectura pública para el bucket exclusivo de pósters. En un entorno de producción se debería restringir SSH y CORS, usar TLS y administrar los secretos con Secrets Manager o SSM.

15 Infraestructura como código

La infraestructura de ASTRA CLOUD se definió con AWS CloudFormation en el archivo `proycloud-infraestructure.yaml`, ubicado en `../infraestructura/aws/`. La plantilla declara la red, los grupos de seguridad, las instancias EC2, el DNS privado, el ALB, los grupos destino,

API Gateway con VPC Link y el bucket de medios. Sus bloques **UserData** preparan las instancias con Docker y Docker Compose, por lo que la topología de despliegue queda versionada.

16 Flujos funcionales

16.1 Autenticación y comunidad

El cliente registra una cuenta o inicia sesión en Identity y recibe un JWT. Community valida el token y relaciona el perfil local mediante el UUID **sub**. A partir de ese perfil, el usuario puede crear clubes, unirse a ellos y abrir salas persistentes.

16.2 Catálogo e interacciones

Catalog entrega películas y metadatos. Al crear una reseña, like, watchlist o visualización, Interaction valida el token, consulta los servicios correspondientes cuando aplica y guarda el documento con su contexto.

16.3 Sesión y analítica

Cinema Session coordina una sesión temporal y distribuye eventos mediante WebSocket. Los contenedores de ingesta extraen las tres bases de datos y cargan archivos CSV a S3; Analytics consulta Athena para entregar indicadores.

17 Pruebas y validación

El proyecto incluye pruebas automatizadas para Community, Interaction y Cinema Session. Community y Cinema Session usan Pytest para validar rutas, permisos y flujos de salas. Interaction emplea `node:test`, Supertest y `mongodb-memory-server` para probar endpoints sin requerir una instancia externa de MongoDB. Los servicios también exponen rutas de salud para la integración y el balanceo. Interaction incorpora `npm run seed`, un script idempotente de carga masiva. Sus valores predeterminados generan 20 000 documentos para reseñas, watchlists e historial, y 25 000 para likes. Las variables `SEED_*` permiten ajustar los escenarios de prueba.

18 Consideraciones de costos

El costo depende principalmente de las cuatro instancias EC2 `t3.medium`, EBS, ALB, API Gateway, S3 y el volumen procesado por Athena. Glue y la transferencia de datos pueden generar cargos cuando se habiliten sus recursos. La separación de ingesta y el uso de Athena permiten ajustar el gasto analítico al volumen de consultas, sin mantener un clúster analítico permanente.

19 Limitaciones actuales

- Cinema Session mantiene su estado en memoria local; para compartirlo entre más réplicas requeriría un almacén como Redis.
- JWT HS256 distribuye un secreto común entre los servicios que validan tokens.
- La plantilla no incluye Auto Scaling Group ni alta disponibilidad para la instancia de base de datos.
- El pipeline genera y carga CSV; la frecuencia de ejecución y los recursos Glue deben configurarse en el despliegue.
- El código fuente del frontend y la configuración de AWS Amplify no están incluidos en este workspace; esta limitación no afecta la disponibilidad de la aplicación web desplegada.

20 Mejoras futuras

Las siguientes iteraciones pueden incorporar Redis para el estado compartido de Cinema Session, un servicio administrado de secretos, monitoreo y alertas con CloudWatch, respaldos, Auto Scaling Groups y CI/CD con pruebas y análisis de imágenes. En analítica, crawlers Glue, particiones y programación de ingestas permitirían operar el pipeline periódicamente.

21 Conclusiones

ASTRA CLOUD implementa una plataforma cinematográfica distribuida mediante seis microservicios especializados. La separación entre identidad, catálogo, comunidad, interacción, sesiones y analítica permite usar tecnologías y mecanismos de persistencia acordes con cada responsabilidad.

La persistencia polígota integra PostgreSQL para las cuentas y el catálogo, MySQL para el modelo social y MongoDB para las interacciones. Docker y Docker Compose hacen reproducible la ejecución, mientras que CloudFormation define la VPC, las EC2, API Gateway, VPC Link, el ALB y S3. El flujo PostgreSQL/MySQL/MongoDB → CSV → S3 → Athena permite realizar consultas analíticas sin trasladar carga a las bases operacionales.

Bibliografía

- [1] Amazon Web Services. Amazon athena user guide, 2026.
- [2] Docker Inc. Docker documentation, 2026.
- [3] FastAPI. Fastapi documentation, 2026.
- [4] Mongoose. Mongoose documentation, 2026.
- [5] PostgreSQL Global Development Group. Postgresql documentation, 2026.
- [6] Proyecto ASTRA CLOUD. Astra cloud source repositories, 2026.
- [7] VMware. Spring boot reference documentation, 2026.